ColPali: Document Retrieval with Vision Language Models ✨

Twitter Antaripa Saha (@doesdatmaksense) on X

Oct 6, 2024

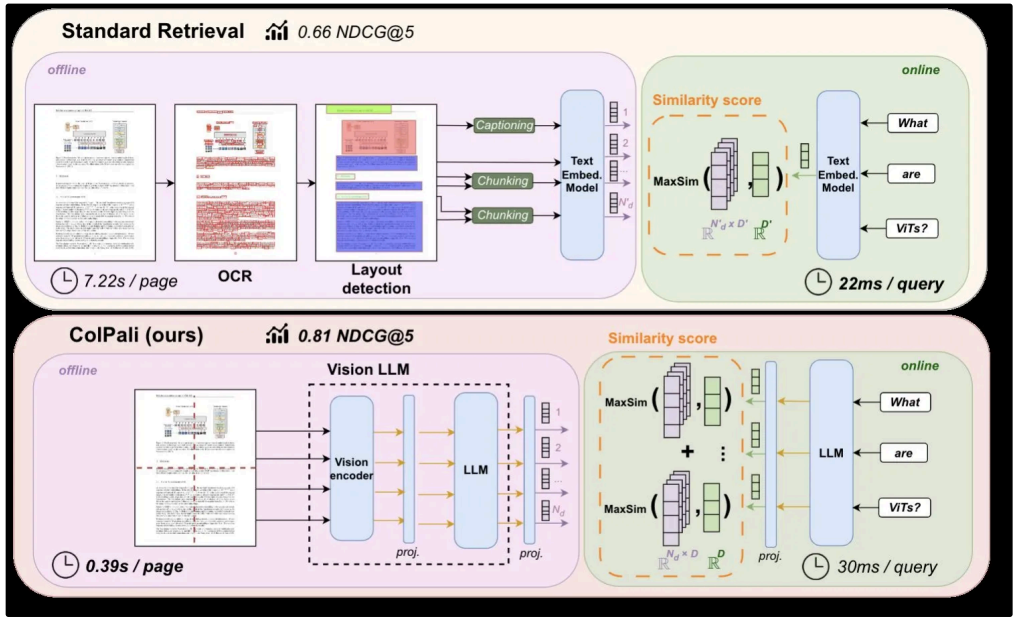
🏹 We will be diving deep into the paper: [Arxiv] ColPali: Efficient Document Retrieval with Vision Language Models

Document retrieval has always been a key component of systems like search engines and information retrieval. Traditional document retrieval methods rely heavily on text-based methods (like OCR and text segmentation), often missing crucial **visual cues** like layouts, images, and tables.

ColPali addresses this by using **Vision-Language Models (VLMs)** to understand and retrieve visually rich documents, capturing both **textual** and **visual information**. ColPali's architecture allows direct encoding of document images into a common embedding space, eliminating the need for time-consuming text extraction and segmentation.

In this blog, we'll explore the technicalities behind ColPali covering the topics:

- Architecture of ColPali
- Training techniques
  - **Contrastive Loss** for query-document matching.
  - Training on **127,460 query-document pairs** (real + synthetic data).
- How it transforms embeddings using techniques like
  - **BiSigLIP** for visual-textual embeddings.
  - **BiPali** for pairing image patches with PaliGemma.
  - **Late Interaction** to achieve state-of-the-art performance.



Before we dive into the technical architecture and training of ColPali, let's walk through the intuition behind how it works.

Intuition Behind ColPali: How It Simplifies Document Retrieval

Step 1: Treating the Document as an Image

Imagine we have a PDF document. Normally, we would extract text from the document using OCR (Optical Character Recognition), segment it into different sections, and then use these segments for searching. ColPali simplifies this process by treating the entire document page as an image, bypassing the need for complex text extraction.

- **Think of it as taking a photograph of each page of the document.** No need to convert the text, handle different languages, or worry about complex layouts. Just treat the page like a picture.

Step 2: Splitting the Image into Patches

Once ColPali has this "image" of the document, it divides the page into small, uniform pieces called **patches (for eg, 16x16** pixels).

- **Each patch captures a tiny portion of the page**—it might contain a few words, a piece of a graph, or part of an image. This division helps the model to focus on small, detailed parts of the document rather than trying to understand the whole page at once.

▼ **How are patches diff from chunking? Will it not lose context in between? [Expand the toggle for a detailed explanation]**

At first glance, it might seem like dividing an image into patches is similar to breaking text into chunks. However, there are several key differences between these two methods, especially in how they handle and preserve context. Let's dive deeper into these differences to understand why patch-based processing in ColPali is more effective for document retrieval compared to traditional text chunking.

Understanding Context Loss in Text Chunking

In traditional text chunking, text is split into smaller chunks based on a certain number of tokens since many models have a limit on the number of tokens they can process at once.

Problem with Context Loss:

- Chunking can split sentences or paragraphs mid-way, losing crucial contex and can result in incomplete information in one chunk and missing context in another.
- Chunking doesn't preserve visual or structural information, like the relationship between headings and their corresponding content or the placement of text in tables or figures.

Visual Example: If you have a document with a heading followed by a table, text chunking might separate the heading and the table, losing the context that the table belongs to that heading.

Patch-Based Image Processing in ColPali

ColPali divides the document image into patches, much like dividing a photo into small squares. Each patch is a fixed-size portion of the image, like a mini-snapshot of that part of the page.

Why Patches are More Effective:

No Loss of Structure: The patches retain the visual structure of the document, preserving its spatial layout. For instance, if a page has two columns of text or a table with rows and columns, each patch maintains its relative position, ensuring that the model understands the overall arrangement of the elements.

Multi-Modal Context: Patches capture both textual and visual information. This includes both visual features (e.g., font styles, colors, boldness) and non-text elements (e.g., figures and graphs).

Positional Awareness: Each patch has a positional embedding that tells the model where it is located on the page, helping the model understand the overall layout.

Step 3: Embedding Creation and Aligning Visual and Textual Information

Each patch is then passed through a Vision Transformer (ViT), which converts them into unique **embeddings**—mathematical codes that represent the content within each patch.

Next, ColPali aligns these visual embeddings with the text of the query by transforming the query into its own set of embeddings. ColPali uses a process called **alignment,** that aligns image path embeddings and text embeddings in the same vector space only then we can compare the similarity between query and document embeddings.

Step 4: Scoring the Relevance - Late Interaction Mechanism

At this point, ColPali has embeddings for both the query and the document. The next challenge is to identify the relevant parts of the document. ColPali uses a process called the **Late Interaction Mechanism**, where each piece of the query is finely matched against every part of the document, scoring and ranking their relevance.

ColPali highlights the most relevant pieces of the document, focusing on the patches that best match the query. This approach enables ColPali to efficiently retrieve relevant information from visually rich documents, capturing both visual and textual data without losing context.

Let's dive deeper now...

ColPali Architecture: A Detailed Exploration

ColPali's architecture is an innovative document retrieval system that handles visual data exclusively. ColPali directly processes document images, representing them as image patches. When a query (in textual form) is introduced, it is embedded into the same shared latent space as the image patches, allowing seamless comparison between the two.

1. Vision-Language Models (VLMs): Aligning Text and Image Embeddings in Shared Space

ColPali leverages **Vision-Language Models (VLMs)** to generate embeddings for both the visual (image patches) and textual content of documents. These image patches are treated as tokens, much like how words are treated in language models. The specific model extended by ColPali is **PaliGemma-3B**, which is known for its efficiency in processing visually rich documents.

PaliGemma-3B combines **Vision Transformers (ViTs)** for visual data with **language models** for textual data, creating a shared representation space for both. The vision transformer breaks down an input image into smaller patches (e.g., 16x16 pixels), and each patch is treated as a separate token. These tokens are then passed through the transformer, similar to how text tokens are processed.

Mathematical Intuition of Image Patch Embedding:

- Let $x_{\text{img}} = \{p_1, p_2, \ldots, p_m\}$ represent the sequence of image patches from a document, where each patch p_i is a segment of the overall document image.

The **Vision Transformer (ViT)** processes each patch independently and generates embeddings:

$$E_{\text{img}} = \text{VisionEncoder}(x_{\text{img}}) = \{e_1, e_2, \ldots, e_m\}$$

where $E_{\text{img}} \in \mathbb{R}^{m \times D}$ and D is the dimensionality of the embedding space.

These embeddings are then compared to the embeddings of the text query, all within the same shared space.

Text Query Embedding:
When a query is provided, ColPali converts it into a sequence of tokens and passes these tokens through a language model. The same VLM architecture is employed to embed the text query into the same latent space as the image patches. This shared space ensures that the query embeddings can be directly compared with the document image embeddings.

Mathematical Intuition:

- Let $x_{\text{query}} = \{q_1, q_2, \ldots, q_n\}$ represent the query tokens, where n is the number of tokens in the query.

The **Text Encoder** then embeds the query:

$$E_{\text{query}} = \text{TextEncoder}(x_{\text{query}}) = \{e'_1, e'_2, \ldots, e'_n\}$$

where $E_{\text{query}} \in \mathbb{R}^{n \times D}$

This means that both the image patches and the text query are represented in the same D-dimensional space, facilitating a direct comparison.

Let's understand in detail the intuition behind alignment of text and image embeddings..

Aligning Text and Image Embeddings in VLM

One of the most fascinating challenges in **vision-language models** is that it must handle two very different types of data: **text** and **images**. These two modalities not only come from different domains but also have **different dimensionalities**. For VLM to work effectively, it needs to **align** these different representations so that the text and image data can be compared in a **common space**.

The Problem: Different Sizes, Different Worlds

VLM model has the ability to **compare** text queries (like "Show me the sales report") with **visual elements** from a document, such as a table or chart. But the embeddings that represent text and image data needs to exist in the same dimensional space. Say,

- **Text Embeddings**: Generated from transformer-based models (such as BERT, GPT), text embeddings typically have sizes like **768 dimensions** or **1024 dimensions**.
- **Image Embeddings**: Produced from **Vision Transformers (ViT)**, image embeddings often have larger dimensions like **1024** or **2048 dimensions**. These embeddings encode the **visual features** of image patches.

Now the question now is, **how do you compare text embeddings of size 768 with image embeddings of size 1024?** We can't, since they are incompatible.

The Solution: Linear Projections

Linear projection is a technique used to bring vectors from different spaces into a common latent space. Here's the mathematical intuition behind it:

Mapping Text Embeddings and Image Embeddings to the Shared Space

Let's say our **text embedding** is of size **768 and** image embedding of size **1024**. We want to transform this into a new space of size **128 (this shared space dimension was used in colpali)** so that it matches the dimensionality of the image embeddings.

This is done using a **linear projection**:

$$\hat{E}_{text} = W_{text} \cdot E_{text} + b_{text}$$

$$\hat{E}_{image} = W_{image} \cdot E_{image} + b_{image}$$

Where:

- E_{text} is your original text embedding (a vector of size 768).
- $W_{text} \in \mathbb{R}^{128 \times 768}$ is the weight matrix that that transforms the text embedding from dimension 768 to 128.
- $b_{text} \in \mathbb{R}^{128}$ is a bias term that shifts the transformed vector appropriately.
- E_{image} is the original image embedding (a vector of size 1024).
- $W_{image} \in \mathbb{R}^{128 \times 1024}$ is the weight matrix that that transforms the image embeddings from dimension 1024 to 128.
- $b_{image} \in \mathbb{R}^{128}$ is a bias term.

This shared space enables cross-modal comparisons, meaning a textual query can retrieve relevant document images based on visual content, such as tables, charts, or other visual elements.

> 🏹 Bonus: You might ask why 128 shared space dimension was used in Colpali? Any specific reason?
>
> To maintain a balance between memory efficiency and computational cost, while minimizing performance drops, we chose a 128-dimensional embedding. This dimension will later be used in a multi-vector reranking process, so smaller is better, but it's essential to strike the right balance for performance. Future versions are planned to offer the option to increase the embedding dimensions.

---

2. Late Interaction Mechanism (for finding similarity between image and query)

The **Late Interaction** mechanism in ColPali is designed to perform **token-level similarity matching** between the text query and document image patches. Unlike traditional retrieval models that reduce everything into a single embedding vector for comparison, **Late Interaction** operates on individual token embeddings, preserving granular details and improving accuracy in retrieval tasks.

The key idea is that for each query token, ColPali finds the **most relevant image patch** in the document. The model then aggregates these relevance scores to compute the overall similarity between the query and the document.

Similarity Scoring with Late Interaction

Late Interaction computes the similarity between **each query token** and **each image patch**. For each query token, it identifies the most relevant image patch using **MaxSim** (maximum similarity). Finally, the total similarity score between the query and the document is the sum of the **maximum similarities** for all query tokens.

Mathematical Intuition Behind Late Interaction:

- **Dot product:** For each query token $e_i\prime$ compute its similarity with each image patch e_j using the dot product. This gives a measure of how well query token $e_i\prime$ aligns with image patch e_j.

$$\text{Similarity}(e_i', e_j) = e_i' \cdot e_j$$

- **MaxSim (Maximum Similarity)**: For each query token $e_i\prime$, find the **most similar image patch** using the maximum similarity across all image patches. This ensures that only the most relevant image patch is considered for each query token.

$$\text{MaxSim}(e_i', E_{\text{img}}) = \max_{j=1}^{m} \langle e_i', e_j \rangle$$

- **Late Interaction Score**: Finally, the total similarity score between the query and the document is computed by summing up the maximum similarities for all query tokens.

$$LI(E_{\text{query}}, E_{\text{img}}) = \sum_{i=1}^{n} \text{MaxSim}(e_i', E_{\text{img}})$$

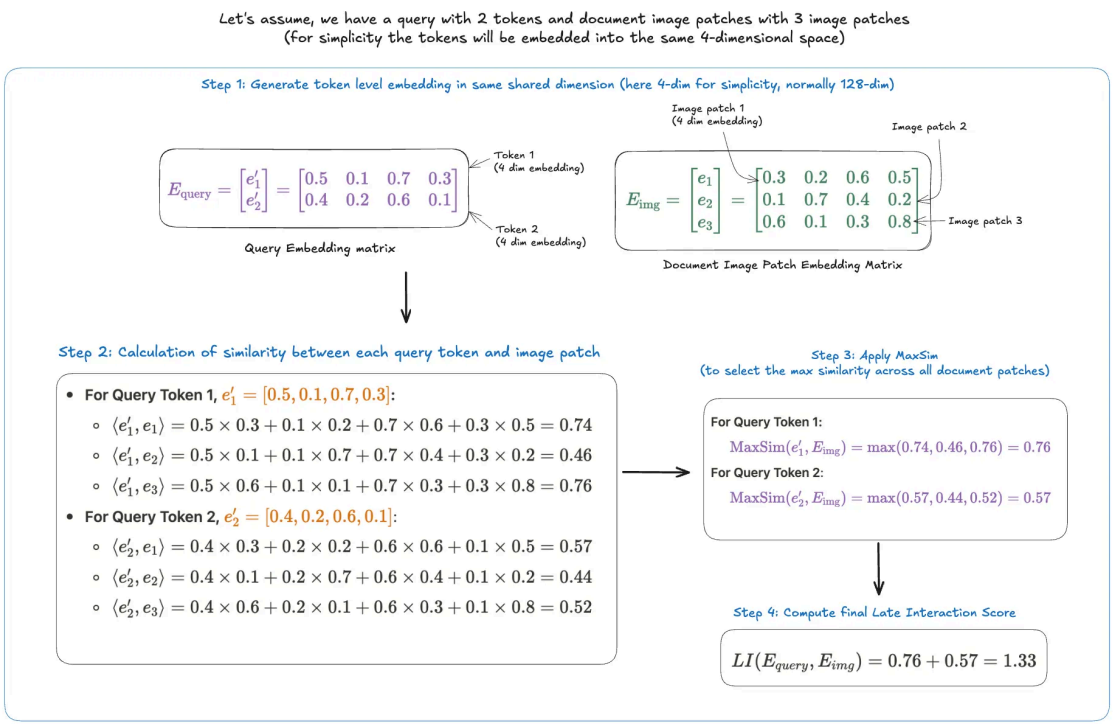


Figure: Visualization of Late Interaction Similarity between query and document patch (By Antaripa)

The term Late Interaction refers to the idea that instead of merging or averaging embeddings from both the query and the document before comparison (as is often done in traditional dense retrieval methods), we **retain token-level interactions** until later in the retrieval process.

Training Techniques and Process in ColPali

Now we will deep-dive into how ColPali is trained to achieve its powerful document retrieval capabilities. The training process focuses on **contrastive learning**, multi-modal alignment (already covered), and the use of techniques like **Low-Rank Adapters (LoRA)**. The dataset is built from both **real and synthetic data**, further enhancing ColPali's generalization capabilities.

1. Contrastive Loss for Query-Document Matching

Here, contrastive loss is used to train the model to correctly associate a **query** with its corresponding **document page** while distinguishing it from other irrelevant pages. The core idea behind constrastive loss is to **pull together** positive query-document pairs and **push apart** negative pairs in the embedding space, enabling the model to learn more discriminative representations for both the query and the document.

Mathematical Intuition of Contrastive Loss

Let's define:

- **Positive Pair**: A query q_k and its corresponding document page d_k.

- **Negative Pair**: A query q_k and a non-relevant document page d_l (where $l \neq k$).

For a given batch of b query-document pairs, this loss encourages the model to maximize the interaction score for correct query-document pairs while minimizing it for incorrect ones.

1. **Late Interaction Similarity**: Using the **Late Interaction** mechanism we discussed earlier, the similarity between the query q_k and its relevant document d_k is computed as:

$$s_k^+ = LI(q_k, d_k)$$

2. **Negative Similarity**: Similarly, the similarity between the query q_k and any other document page d_l is computed using the **max similarity** from Late Interaction:

$$s_k^- = \max_{l \neq k} LI(q_k, d_l)$$

3. **In-Batch Contrastive Loss**: The final **contrastive loss** for a batch of size b is calculated using softmaxed cross-entropy between the positive score and all the negative scores:

$$Loss(L) = -\frac{1}{b} \sum_{k=1}^{b} \log \frac{\exp(s_k^+)}{\sum_{l=1}^{b} \exp(s_l^-)}$$

💡 But why these -1/b, log, exp, etc. etc.? If that's not making sense to you, then please expand and understand the intuition behind the formula.

▶ **Breaking down the contrastive loss formula and reasoning behind it in detail. [Please expand]**

2. Dataset and Data Augmentation

Training Dataset:

ColPali is trained on a massive dataset of **127,460 query-document pairs** composed of both **real and synthetic data**:

- **63% of the data** comes from real, publicly available academic datasets.
- **37% of the data** is synthetic, created by web-crawling PDF documents and generating queries using a **Vision-Language Model (VLM)** called **Claude-3 Sonnet**.

This mix of real and synthetic data helps ColPali generalize well to unseen documents. Interestingly, the training dataset is **fully English**, but ColPali shows strong **zero-shot generalization** to non-English languages.

Query Augmentation:

Inspired by **ColBERT**, ColPali uses a technique called **Query Augmentation**, where 5 **<unused> tokens** are appended to the query during training. These tokens are placeholders, and while they don't have any predefined meaning, they can act as **learnable parameters** that help the model adjust its attention during the retrieval process. These unused tokens allow the model to:

- **Expand** the query dynamically.
- **Re-weight** query tokens, making it easier for the model to learn which parts of the query are most important for retrieval.

Mathematical Intuition:

- Let the original query be $q = \{q_1, q_2, \ldots, q_n\}$
- We append 5 **<unused> tokens** to this query, so the final query becomes:

$$q' = \{q_1, q_2, \ldots, q_n, <\text{unused1}>, <\text{unused2}>, \ldots, <\text{unused5}>\}$$

These unused tokens are treated like regular query tokens during training but allow the model to adjust the importance of certain parts of the query during retrieval.

I won't further explain query augmentation in detail, please refer ColBERT's paper for more detailed explanation: ColBERT paper

---

3. Low-Rank Adapters (LoRA) for Efficient Training

Training large models from scratch is computationally expensive and often impractical. To make the training process more efficient, ColPali uses **Low-Rank Adapters (LoRA)**, a technique that allows fine-tuning a small subset of the model's parameters without requiring the full model to be updated.

How LoRA Works:

In the transformer layers of the language model:

- **LoRA** adds a **low-rank matrix** to the attention weights and **only fine-tunes** these low-rank matrices during training.
- This drastically reduces the number of trainable parameters, making fine-tuning more efficient.

Again, not going in-depth into LoRA here (it's out of scope of this blog)

BiSigLIP and BiPali: Embedding Techniques

BiSigLIP: Visual-Textual Embedding Model

ColPali builds on **SigLIP**, a vision-language bi-encoder model. SigLIP is pre-trained on **WebLI**, a massive corpus of billions of **image-text pairs**. In ColPali, SigLIP is fine-tuned on a **document retrieval dataset**, which allows it to handle the **visual and textual elements** of documents more effectively.

1. **SigLIP as a Bi-Encoder**: SigLIP generates embeddings for both **image patches** and **text** and aligns them in the same latent space, enabling cross-modal comparison between **text queries** and **document images**.

2. **BiSigLIP Fine-Tuning**: ColPali fine-tunes SigLIP on a **document-oriented dataset** to improve its retrieval performance.

BiPali: Pairing Image Patches with PaliGemma

In the **BiPali** model, **SigLIP-generated image patch embeddings** are passed through a **language model (PaliGemma)** to obtain **contextualized output embeddings**. This enhances the embeddings by adding **language-based context**, which helps in tasks that require understanding the relationship between text and images.

- **Pooling Operation**: The image patch embeddings are **average pooled** to create a single dense vector representing the entire document. Remember, **BiPali** initially explored **pooling** as a way to create a single dense vector from image patch embeddings. But, **ColPali** does **not use pooling**.

Late Interaction

The **ColPali** model further extends this by introducing token-level interaction between text and image embeddings, which drastically improves performance on complex visual tasks.

As we near the conclusion of this blog, wouldn't you be curious to explore the outcomes of Colpali's experiments with various architectures and techniques? Let's dive in!

Results and Lessons from ColPali's Iterative Development

In constructing **ColPali**, the authors iteratively built and improved upon various models, starting with an off-the-shelf **SigLIP** model, followed by pairing SigLIP with a language model (**PaliGemma**) and finally adding **Late Interaction** (as seen above). Each iteration provided insights into the model's performance across different document retrieval tasks, particularly for documents with complex visual elements like tables and figures.

Improvements with BiSigLIP

When fine-tuned on the **document retrieval dataset**, **BiSigLIP** showed significant improvements across various document retrieval tasks:

- **ArxivQA (figure retrieval)**: Focused on retrieving figures from academic papers.
- **TabFQuAD (table retrieval)**: Tasked with retrieving tables from documents.

By further fine-tuning SigLIP's text and image encoders on this **document-specific dataset**, ColPali achieved improved performance for tasks requiring understanding both **textual and visual information**.

Performance of BiPali

After fine-tuning on the training dataset, **BiPali** showed a slight decrease in performance for **English document retrieval tasks** compared to **BiSigLIP**. This is likely due to the fact that **PaliGemma** was not originally trained for **contrastive matching tasks**, but rather for **next token prediction**. The authors' **contrastive fine-tuning** on 100K images was significantly smaller than SigLIP's original contrastive training, leading to **weaker performance in English retrieval tasks**.

- **Why BiPali Shines in Multilingual Tasks**

  Despite the slight drop in English performance, **BiPali** showed **notable improvements in French tasks**, indicating that **PaliGemma's multilingual pretraining** helps with **multilingual document retrieval**. Interestingly, although the training dataset did not contain any non-English samples, **PaliGemma's LLM** (Gemma-2B) was able to handle multilingual data, resulting in better cross-lingual generalization.

Why Late Interaction Improves Performance

By focusing on the **most relevant document patches** for each query token, **Late Interaction** enables ColPali to excel in tasks that require **detailed understanding of both text and visual elements**. This was especially evident in more **visually complex tasks**, such as:

- **InfographicVQA** (infographic retrieval).
- **ArxivQA** (figure retrieval).
- **TabFQuAD** (table retrieval).

ColPali outperformed baselines such as **Unstructured and captioning-based models**, as well as all evaluated **text-image embedding models**. This stark improvement in visually complex tasks demonstrates the power of **Late Interaction** in multimodal retrieval.

Lessons from Negative Results: ColSigLIP and BiSigLIP with Late Interaction

ColSigLIP:

A version of **BiSigLIP** with **Late Interaction** (ColSigLIP) was also tested but **performed poorly**. The authors attribute this to the fact that in **SigLIP's pre-training**, only a **pooled latent representation** is used in the contrastive loss, which **does not optimize individual patch and token embeddings** as effectively as ColPali.

BiSigLIP + PaliGemma:

The authors also experimented with using the **text representations from PaliGemma** and the **image representations from SigLIP**. However, this variant also performed poorly, likely due to a **misalignment** between the **SigLIP embeddings** and the **Gemma embeddings** after PaliGemma fine-tuning.

Querying Latencies and Memory Footprint